Probeklausur (überarbeitet)

Bearbeitungszeit 90 Minuten

Name	**Probeklausur**
Vorname	**.**
Matrikelnummer	**.**
Datum	**22.01.2008**

Bei der Klausur sind **keine Hilfsmittel** (Skripten, Taschenrechner, etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten Blätter für Ihre Antworten. Schreiben Sie mit blauem oder schwarzem Kugelschreiber, Füller, Bleistift, etc. ist nicht erlaubt.

1 Abstrakte Beschreibung	14	
2 Logik	10	
3 Zahlen	14	
4 Algorithmen	22	
5 Wachstum von Funktionen	16	
6 Abstrakte Datentype	10	
7 Java	04	
Summe	**90**	

Note	

1 Abstrakte Beschreibung (1/2)

1. Gegeben ist der folgenden Java-Quelltext der Methoden `fac(n)` zur Berechnung der Fakultät von n.

```
1  public int fac(int n) {
2    int f = 1;
3    while (n > 1) {
4      f = f * n;
5      n = n - 1;
6    }
7    return f;
8  }
```

   Zeichnen Sie das Flussdiagramm der Methoden `fac`. (4 Punkte)

1 Abstrakte Beschreibung (2/2)

2. Geben Sie über dem Alphabet der Ziffern von 0 bis 9 eine EBNF (erweiterte Backus-Naur-Form) Beschreibung für die Sprache aller natürlichen Zahlen an, die durch 4 teilbar sind. (4 Punkte)

3. Es sind reguläre Audrücke definiert für das leere Wort (ε), die Konkatenation (Hintereinanderstellen), die Alternative ($|$) und beliebig viele Vorkommen ($*$).

   Für reguläre Ausdrücke gilt Folgendes.

   - $*$ bindet stärker als die Konkatenation, Beispiel. $rs* = r(s*)$.
   - Die Konkatenation bindet stärker als $|$. Beispiel. $(rs|ab) = ((rs)|(ab))$.

   Der reguläre Ausdruck $r?$ soll kein oder ein Vorkommen, der reguläre Ausdruck $r+$ soll beliebig viele, aber mindestens ein, Vorkommen von r beschreiben.

   Geben Sie Definitonen von $r?$ und $r+$ mit den bereits definierten regulären Ausdrücken an. (6 Punkte)

2 Logik

Folgende Gesetze (Rechenregeln) für Operatoren auf Wahrheitswerten können als bekannt vorausgesetzt werden.

Neutralitäts-	Extremal-	Komplementär	Kommutativ-	Idempotenz-
$x \vee 0 = x$	$x \vee 1 = 1$	$x \vee \neg x = 1$	$x \vee y = y \vee x$	$x \vee x = x$
$x \wedge 1 = x$	$x \wedge 0 = 0$	$x \wedge \neg x = 0$	$x \wedge y = y \wedge x$	$x \wedge x = x$
				$\neg\neg x = x$

Assoziativ-	Distributiv-
$x \vee (y \vee z) = (x \vee y) \vee z$	$x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$
$x \wedge (y \wedge z) = (x \wedge y) \wedge z$	$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$

Hinweis. Der $\neg$ Operator bindet stärker als die Operatoren $\vee$, $\wedge$, d.h. $\neg\ x \vee y = (\neg x) \vee y$ und $\neg\ x \wedge y = (\neg x) \wedge y$. Die Operatoren $\vee$, $\wedge$ binden gleichstark, deshalb sind hier Klammern, die die richtige Auswertungsreihenfolge bestimmen, zwingend erforderlich (Beispiel, Distributivgesetz).

1. Zeigen Sie, mit Hilfe einer ausführlichen Wahrheitswertetafel, dass folgendes De-Morgansche-Gesetz gilt. (4 Punkte)

$$\neg(x \vee y) = \neg x \wedge \neg y$$

2. Betrachten Sie die Absorptionsgesetze.

$$x \vee (x \wedge y) = x$$
$$x \wedge (x \vee y) = x$$

   a) Zeigen Sie, durch Anwendung der passenden bekannten Gesetze, dass aus der Gültigkeit eines Absorptionsgesetze das andere folgt. Geben Sie bei jedem Schritt das angewendete Gesetz an. (3 Punkte)

   b) Zeigen Sie die Gültigkeit eines Absorptionsgesetzes durch Anwendung der passenden bekannten Gesetze. Geben Sie bei jedem Schritt das angewendete Gesetz an. (3 Punkte)

      Hinweis. Beginnen Sie, indem Sie $\wedge\neg x$ auf beide Seiten der Gleichung anwenden.

3 Zahlen

1. Rechnen Sie von einem Zahlensytem in das andere um. Der Rechenweg ist Teil der Lösung.

   a) $(377)_8 = (?)_{10}$ (2 Punkte)

   b) $(3200)_{10} = (?)_{16}$ (2 Punkte)

   c) $(1001011001)_2 = (?)_8$ (2 Punkte)

   d) $(E2A0)_{16} = (?)_2$ (2 Punkte)

2. Betrachten Sie vorzeichenbehaftete Dualzahlen der Länge n Bit, wobei für die Darstellung der negativen Zahlen das Basiskomplement verwendet wird.

   a) Erläutern Sie, wie zwei solche vorzeichenbehafteten Dualzahlen addiert werden. (4 Punkt)

   b) Rechnen Sie exemplarisch $\texttt{01110}_2 - \texttt{00111}_2$ mit $n = 5$. (2 Punkt)

4 Algorithmen (1/2)

1. Behauptung. Die Anzahl der Teile der Länge 1 oder 2, in die ein Faden der positiven ganzzahligen Länge n zerschnitten werden kann, wird durch folgende rekursive Formel bestimmt.

$$T(1) = 1$$
$$T(2) = 2$$
$$T(n) = T(n-1) + T(n-2)$$

   a) Zeigen Sie mit vollständiger Induktion die Korrektheit der Behauptung. | (6 Punkte)

   b) Geben Sie in Pseudo-/Java-Code einen Algorithmus zur iterativen Berechung der Anzahl der Teile der Länge 1 oder 2, in die ein Faden der positiven ganzzahligen Länge n zerschnitten werden kann, an. (6 Punkte)

4 Algorithmen (2/2)

2. Betrachten sie den Java-Programmtext der Funktion `f`.

```
1  public boolean f(int x) {
2      boolean b = true;
3      if ( x < 1 )
4          b = false;
5      else if ( x > 1 ) {
6          int t = 2;
7          while ( t*t <= x ) {
8              if ( b && ((x%t) == 0) )
9                  b = false;
10             t++;
11         }
12     }
13     return b;
14 }
```

a) Beschreiben Sie in maximal zwei Sätzen die Funktion der Methode `f`. (4 Punkte)

Hinweis. Welche Eigenschaft der übergebenen Zahl kann aus dem Rückgabewert der Methode gefolgert werden.

b) Definieren Sie für die Schleife Vorbedingung, Nachbedingung und Schleifeninvariante. Beweisen Sie mit deren Hilfe die Korrektheit Ihrer Aussage über die Funktion der Methode `f`. (6 Punkte)

Hinweis. Berücksichtigen Sie auch die Sonderfälle.

5 Wachstum von Funktionen

1. Sei $f(n) \in \mathcal{O}(g(n))$ und $a > 0$.

   Beweisen Sie $f(n) \in \mathcal{O}(a \cdot g(n))$. (4 Punkte)

2. Die folgende Tabelle definiert die Primitiven *Zuweisung*, *Sequenz*, *Alternative* und *Schleife* zur Zerlegung von Methoden. Berechnungen und Vergleiche ohne Zuweisung bleiben unberücksichtigt. Weiterhin ordnet die Tabelle jeder Primitive P eine Kostenmaß $T(P)$ zu.

Primitive	*Beispiel für P*	*Kosten T(P)*
Zuweisung	`a = b + c;`	1
Sequenz	P_1; P_2; ⋮ P_m;	$T(P_1) + T(P_2) + \ldots + T(P_m)$

Alternative	`if (<bedingung>)`
	  P_1;
	`else`
	  P_2;
		$\max(T(P_1), T(P_2))$

Schleife	`for(i=0; i<k; i++)` P_1;	$k \cdot T(P_1)$

   Hinweis. Die Rückgabe eines Funktionswertes (`return`) ist eine Zuweisung.

   Betrachten Sie folgende Methode `armstrong` zum Test ob die übergebene Zahl eine Armstrong-Zahl ist.

```
1  public boolean armstrong(int x) {
2      int summe = 0;
3      int zahl = x;
4      if ( zahl > 0 ) {
5          do {
6              int ziffer = zahl % 10;
7              summe = summe + ziffer*ziffer*ziffer;
8              zahl = zahl/10;
9          } while (zahl > 0);
10     }
11     return (x == summe);
12 }
```

   a) Geben sie, basierend auf obigem Kostenmaß, eine Laufzeitfunktion $T_a(n)$ für die Methode `armstrong` an. (6 Punkte)

   b) Geben Sie, mit Herleitung, die asymptotische Laufzeit der Methode `armstrong` in $\mathcal{O}$-Notation an. (6 Punkte)

6 Abstrakte Datentypen

Ihnen steht eine Klasse `CharStack`, die einen Kellerspeicher für Zeichen implementiert, mit folgenden Methoden zur Verfügung.

Methode	*Funktion*
`void create`	Konstruktor, erzeugt einen leeren Stapel
`boolean empty`	Liefert *`true`*, wenn der Stapel leer ist, sonst *`false`*.
`void push(char ch)`	Legt das Zeichen `ch` oben auf den Stapel.
`char pop()`	Liefert das oberste Zeichen zurück und entfernt es vom Stapel, ist der Stapel leer wird `null` zurückgeliefert.

Schreiben Sie in Pseudo-/Java-Code eine Methode, die, mit Hilfe der Klasse `CharStack`, einen übergebenen String darauf testet, ob er ein Palindrom ist. (10 Punkte)

Hinweis. Ein Palindrom ist ein Wort bzw. eine Zeichenkette, die vorwärts und rückwärtsgelesen, genau gleich ist. Beispiele. `LAGERREGAL` oder `p#?& &!& &?#p`.

7 Java

Betrachten Sie folgendes Java-Programmfragment.

```
1  for ( i=0; i < n; i++) {
2      if ( a[i] == x)
3          break;
4      else
5          a[i] = 0;
6  }
```

Schreiben Sie das Java-Programmfragment so um, dass anstelle der `for`– eine `do-while`–Schleife und keine `break`–Anweisung benutzt wird. (4 Punkte)

Hinweis. Auch ein leeres Feld ($n = 0$) muß berücksichtigt werden.